AIM2 Preparation
Week 2 - Day 4: EDA / Data Analysis

5-Minute Review Notes | Pandas Exploratory Data Analysis

Day 4 Goal

Explore a dataset with Pandas, summarize its structure and statistics, compare groups, identify top/bottom values, and study relationships between numerical columns.

EDA: What Are We Doing?

Exploratory Data Analysis (EDA) means inspecting and summarizing data before building models. The goal is to understand what the dataset contains, how values are distributed, how groups differ, and whether variables appear related.

Typical flow: Inspect -> Summarize -> Compare -> Group -> Find patterns -> Interpret

1. Inspecting the Dataset

```
print(df.shape)
print(df.columns)
print(df.columns.tolist())
print(df.dtypes)
```

- df.shape -> (rows, columns)
- df.columns -> column labels as a Pandas Index
- df.columns.tolist() -> column labels as a normal Python list
- df.dtypes -> data type of each column

2. describe() and Summary Statistics

```
ds = df.describe()
print(ds)
```

For numerical columns, describe() gives count, mean, standard deviation, minimum, quartiles, median (50%), and maximum.

Statistic	Meaning
mean	Add all values and divide by the number of values.
median / 50%	Middle value after sorting the data.
std	How spread out values are around the mean.
min / max	Smallest / largest value.

25% / 75%	First and third quartiles.

Accessing Statistics

```
print(ds.loc["mean", "Math"])
print(ds.loc["50%", "Math"])
print(ds.loc["std", "Math"])

print(df["Math"].mean())
print(df["Math"].median())
print(df["Math"].std())
```

Using .loc with labels is usually clearer than relying on fixed row/column positions with .iloc when the statistic names are known.

3. Standard Deviation

Low standard deviation -> values are clustered/close to the mean.

High standard deviation -> values are more spread out from the mean.

Memory shortcut: LOW std = close together | HIGH std = spread apart

4. Unique Values and Counts

```
df["City"].unique()
df["City"].nunique()
df["City"].value_counts()
```

- unique() -> returns which unique values exist.
- nunique() -> returns how many unique values exist.
- value_counts() -> counts how many rows belong to each value/category.

Percentages with value_counts()

```
df["City"].value_counts(normalize=True) * 100
```

normalize=True returns proportions instead of raw counts. Multiplying by 100 converts them to percentages.

5. Calculated Columns

```
df["Average"] = df[["Math", "Python", "AI"]].mean(axis=1)
```

axis=1 means calculate across columns for each row. Here, each student's Math, Python and AI scores are averaged.

6. Sorting and Finding Top/Bottom Rows

```
df.sort_values("Average", ascending=False)

top_index = df["Average"].idxmax()
low_index = df["Average"].idxmin()

df.loc[top_index]
df.loc[low_index]
df.loc[top_index, "Name"]
df.loc[top_index, ["Name", "Average"]]
```

- sort_values(..., ascending=False) -> highest to lowest.
- idxmax() -> index label of the maximum value.
- idxmin() -> index label of the minimum value.
- loc[index] -> retrieve the corresponding row or selected columns.

7. GroupBy - Analyze Categories

```
df.groupby("City")["Math"].mean()
```

Interpretation: group rows by City -> select Math scores inside each city group -> calculate the mean for each city.

Important distinction: df["Math"].mean() gives one overall Math average for all students, while df.groupby("City")["Math"].mean() gives a separate Math average for each city.

Multiple Columns with groupby()

```
df.groupby("City")[["Math", "Python", "AI"]].mean()
```

This produces subject averages separately for every city.

8. Multiple Aggregations with agg()

```
df.groupby("City")["Math"].agg(["mean", "min", "max"])
```

agg() lets us calculate several summary statistics for each group in one operation.

9. Correlation

```
df[["Math", "Python", "AI"]].corr()
```

Correlation measures the strength and direction of a linear relationship between numerical variables.

Correlation	Interpretation
Close to +1	Strong positive relationship
Close to 0	Weak/no linear relationship
Close to -1	Strong negative relationship

Critical rule: Correlation does not prove causation. A value such as 0.95 means strong correlation, not that one variable caused the other.

10. Day 4 Dataset Insights

- Math mean: 80.625; median: 81.5; standard deviation: about 11.01.
- London had the highest average Math score (86.0) in the exercise dataset.
- Priya had the highest overall Average; Amit had the lowest.
- Math, Python and AI scores showed very strong positive correlations in this small dataset.

Day 4 Mini-Project - Student Performance EDA

1. Dataset overview: number of students, columns, unique cities.
2. Overall performance: Math, Python, AI and overall averages.
3. Top/bottom performance: highest, lowest and top 3 students.
4. City analysis: student counts, subject averages and Math mean/min/max.
5. Relationships: correlation matrix for Math, Python and AI.

Quick Syntax Cheat Sheet

```
df.describe()
df["Math"].mean()
df["Math"].median()
df["Math"].std()
df["City"].unique()
df["City"].nunique()
df["City"].value_counts()
df["City"].value_counts(normalize=True) * 100
df.sort_values("Average", ascending=False)
df["Average"].idxmax()
df["Average"].idxmin()
df.groupby("City")["Math"].mean()
df.groupby("City")["Math"].agg(["mean", "min", "max"])
df[["Math", "Python", "AI"]].corr()
```

Key Takeaways

- Mean is the arithmetic average; median is the middle ordered value.
- Standard deviation tells us how spread out values are around the mean.
- unique() returns the categories; nunique() counts them.
- groupby() lets us compare statistics between categories.
- idxmax()/idxmin() locate the rows containing extreme values.
- agg() performs multiple summary calculations per group.
- corr() measures association between numerical variables, not causation.
- EDA turns raw data into understandable summaries and questions for deeper analysis.

Progress

Week 2 Day 4 - EDA / Data Analysis: COMPLETE

Next completed topic: Week 2 Day 5 - Matplotlib Visualization.